The workflow for git-based z/OS DevOps

Recommended by the DAT Team

Ian J Mitchell Dennis Behm Mathieu Dalbin

Table of contents

0.1 Agenda . . . 2

1 Aims and Assumptions **2**
1.1 No baggage . . . 2
1.2 Scaling . . . 3
1.3 Planning . . . 3

2 The Branching Strategy **3**
2.1 Starting Simple . . . 3
2.2 Starting Simple . . . 4
2.3 Preparing to Merge into `main` . . . 4
2.4 Merging into `main` . . . 5
2.5 Before you ask... no, there is no *Production* branch . . . 5
2.6 Testing a release candidate . . . 6
2.7 Deploying to production . . . 6
2.8 Release maintenance branches . . . 7
2.9 Scaling up . . . 8
2.10 Integration branches . . . 9
2.11 Convention over configuration . . . 9

3 Workflows supported by the strategy **10**
3.1 The types of workflows . . . 10
3.2 Features for the next planned release . . . 10
3.3 Features for the next planned release . . . 11
3.4 Fix a production defect . . . 11
3.5 Fix a production defect . . . 12
3.6 Fix a production defect . . . 13
3.7 Using the Epic branch workflow . . . 13
3.8 Using the Epic branch workflow . . . 13

3.9 Using the Epic branch workflow 14
3.10 Epic branch workflow 14

4 Pipeline implementations **14**
4.1 Pipeline implementation conventions & principles 14
4.2 Basic Build pipeline characteristics 14
4.3 Release pipeline 15

5 Collateral **15**
5.1 References 15

0.1 Agenda

- Aims and Assumptions
- Fundamentals of the branching strategy
- Supported workflows
- Implementing pipelines

1 Aims and Assumptions

Some aims and assumptions that guide our recommendations...

1.1 No baggage

Well... *"travel light"* perhaps!

- Are we *prescriptive* or just *opinionated*?
  - Aim to be *git-native*!
- We start with a recommendation
  - Confidently
- We question everything
  - **YAGNI** - "you aren't gonna need it"
- We strive for simplicity
  - For each user's experience

1.2 Scaling

- The workflow and branching scheme should both scale up and scale down.
  - Small teams with simple and infrequent changes will be able to easily understand, adopt, and have a good experience.
  - Large, busy teams with many concurrent activities will be able to plan, track, and execute with maximum agility using the same fundamental principles.

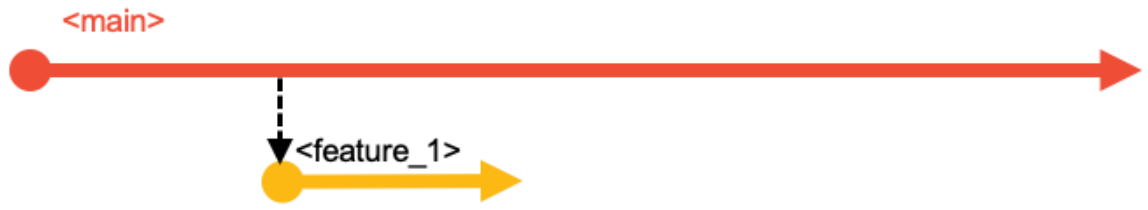
1.3 Planning

- Planning and design activities as well as code development aim to align to a regular release cadence.
- The cadence of the next planned release is defined by the application team.
- There is no magic answer to managing large numbers of "in-flight" changes, so planning assumptions should aim as much as possible to complete changes quickly, ideally within one release cycle.
  - DevOps/Agile practices to break down larger changes into sets of **smaller, incremental** deliverables that can each be completed within an iteration.
  - This reduces the number of "in-flight" changes, and allows the team to deliver **value** more quickly while still building towards a larger development goal.
- We know it is sometimes unavoidable for work to take longer than one release cycle and we accommodate that as a variant of the base workflow.

2 The Branching Strategy

2.1 Starting Simple

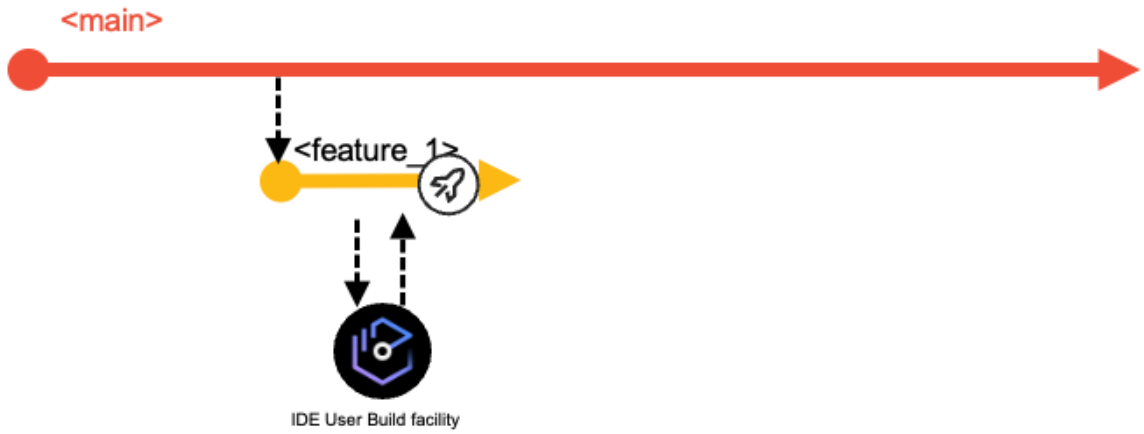
Every change starts in a branch.



- Developers work in the branch to make changes, perform user builds and unit tests.
- A branch holds multiple commits (changes to multiple files).

2.2 Starting Simple

Every change starts in a branch.



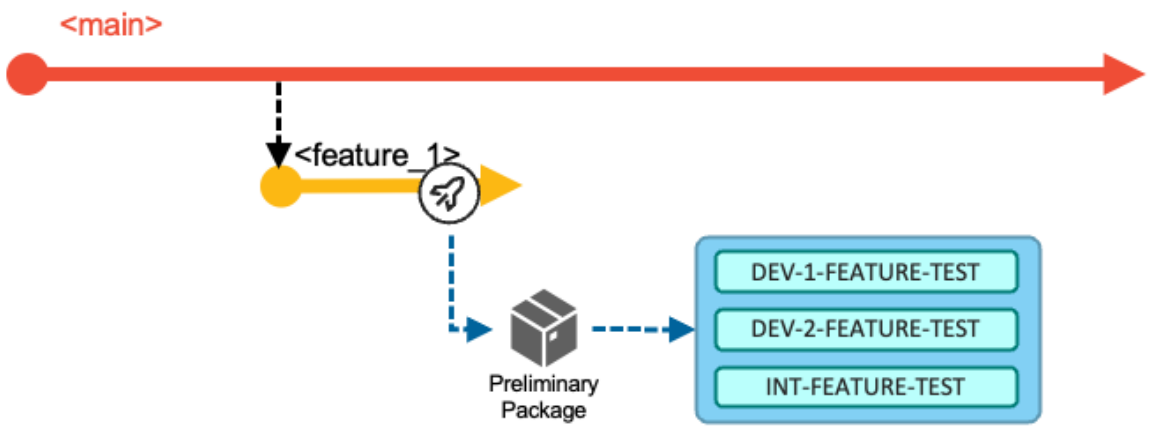
These changes on these branches are

- built,
- tested,
- reviewed and
- approved before merging to `main`.

2.3 Preparing to Merge into `main`

Feature Team/Developers will:

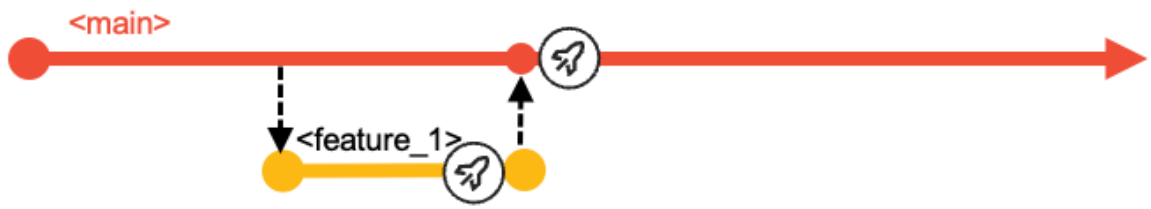
- Build
  - Builds may be done to any commit on any branch
  - Feature branch **must** build cleanly for a Pull Request
- Test
  - To prove quality of the changes in their feature branch



2.4 Merging into `main`

Create a Pull Request (PR) to signal to Team Leaders/Release Controllers to:

- Review
  - Code and Test results
- Approve
  - Safeguard the quality of `main`



`main` and other long-lived branches are *protected* (not everyone can `push` or `merge` to them).

2.5 Before you ask... no, there is no *Production* branch

CI/CD decouples the building and deploying to test environments and production.

We have no branches named `prod` (or `test` or `QA`)

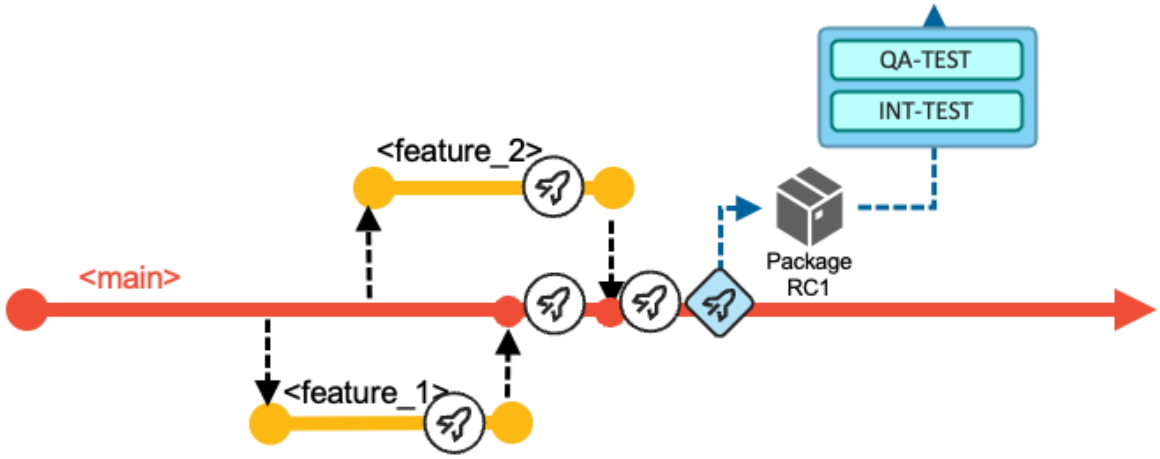
- Those are *environments* to which builds can be deployed

- Such extra branches:
  - are unnecessary
  - cause ambiguity
  - impose merging and building overheads
- Deployment manager is maintaining what is deployed where and provides traceability from developer to production deployment

2.6 Testing a release candidate

Any point in the history of `main` can be declared a *release candidate*.

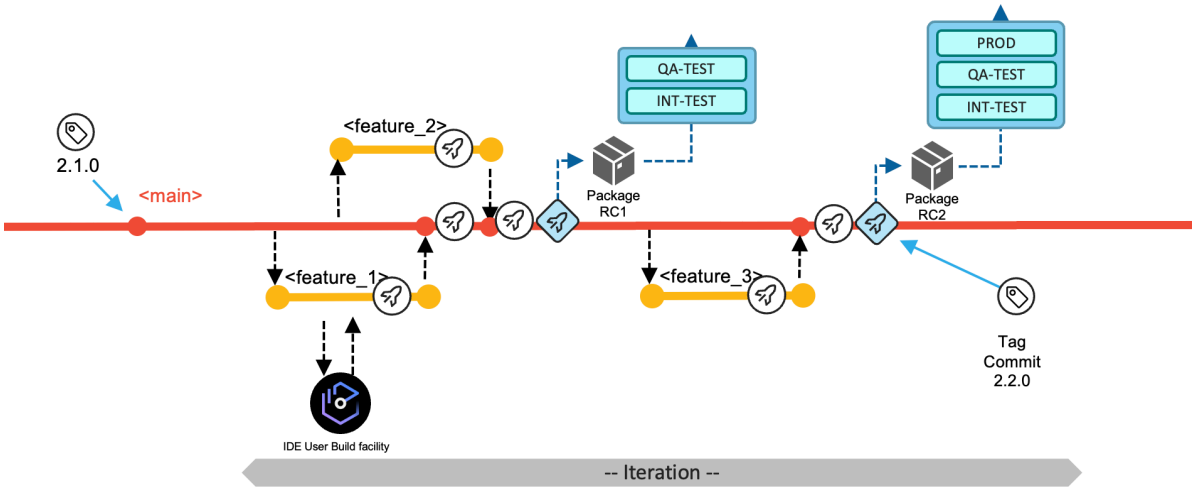
- Build a *release candidate* package
- Deploy it
- Test it



Tag the commit (point in `main`'s history) with a release name.

2.7 Deploying to production

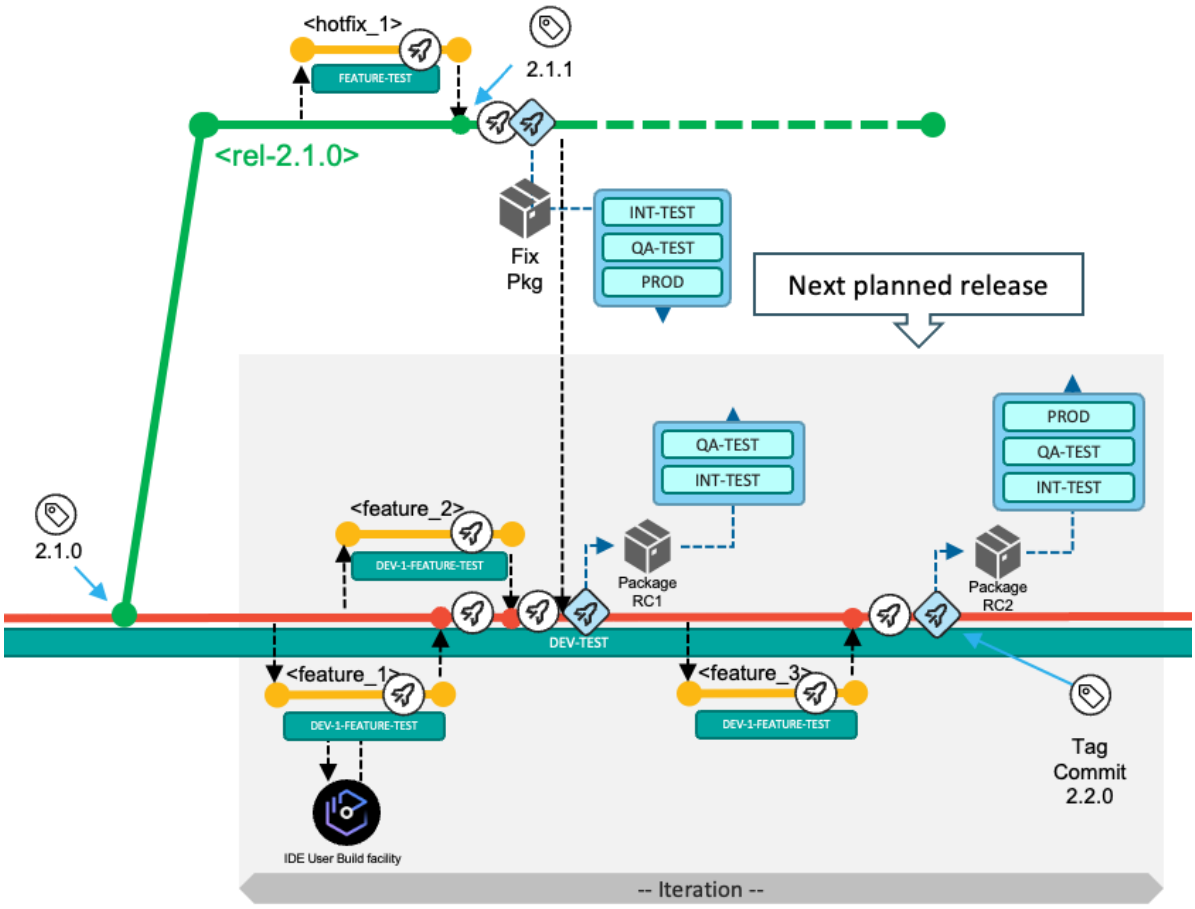
When all the committed work items for the next planned release are ready to be shipped, and all quality gates have passed successfully and the required approvals have been issued by the appropriate reviewers, the release package and be deployed to production.



Tag the commit (point in `main`'s history) with a release name.

2.8 Release maintenance branches

A *release maintenance* branch will be used if *hot-fixes* must be developed and delivered.



2.9 Scaling up

Concurrent *feature* branches scale very well, but assume short cycle times.

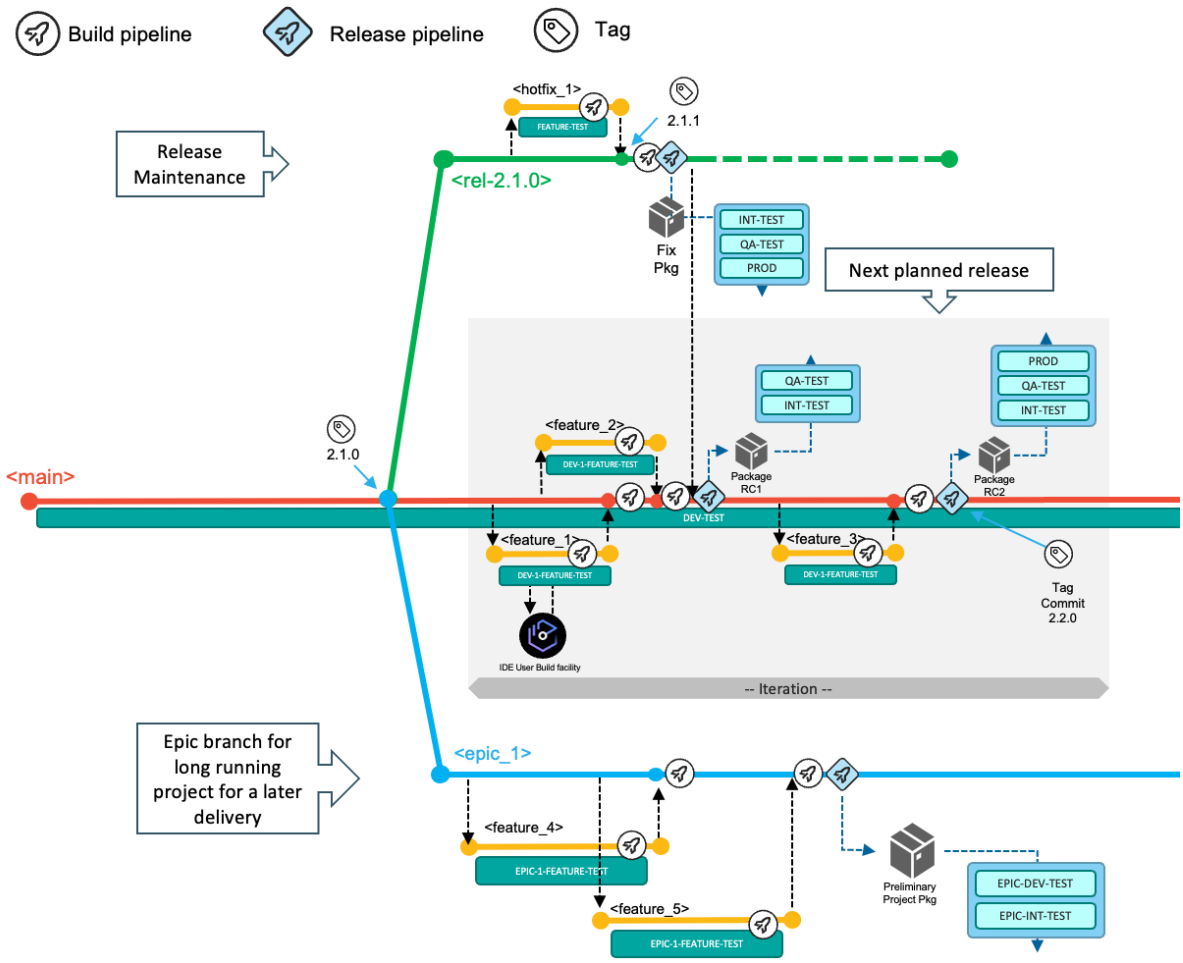
- Ideally live within a release delivery cycle
- But no big deal if they don't

Epic branches can collect multiple features

- Before going to `main`
- When the delivery is planned beyond the next release

(*Epic* branches are a form of *integration* branch.)

2.10 Integration branches



2.11 Convention over configuration

The principles are more important that the tools and names.

Naming conventions - making purpose obvious:

- `main` : single source of truth. The only long-living branch.
- `release/rel-2.0.1` : explicit versioning numbering to identify releases maintenance branch.
- `epic/47-ai-fraud-detection` : development initiative identified by epic id and description.

- `feature/<jira-id|servicenow-id>/new-mortgage-calculation` : references to other planning tools for new features for the next planned release.
- `feature/47-ai-fraud-detection/refactor-mortgage-calculation`: feature implemented for development initiative
- `hotfix/rel-2.0.1/fix-mortgage-calc`: fix implemented for release rel-2.0.1

Names of branches and tags flow through to builds and deployments.

3 Workflows supported by the strategy

How do teams work with and benefit from the branch strategy?

3.1 The types of workflows

1) Work and focus on the **next planned release** via the `main` branch. After planning the work items for the next release, the development team is adding changes to the main branch.

2) **Resolution of a production problem** in the currently-released version of the application by leveraging a release maintenance branch that is used for maintenance purposes,

3) **concurrent development activities for significant development initiatives**, which include multiple planned work items for a later delivery (including starting development of a future release) by creating an epic branch from a commit point in the history of main.

The following narratives complement the structure.

3.2 Features for the next planned release

Developer activities:

1. Take item from backlog
2. Create feature branch
3. Code locally
4. Build with DBB User Build
5. Commit into feature branch, build and test feature functionality
6. Create Pull Request for review and approval

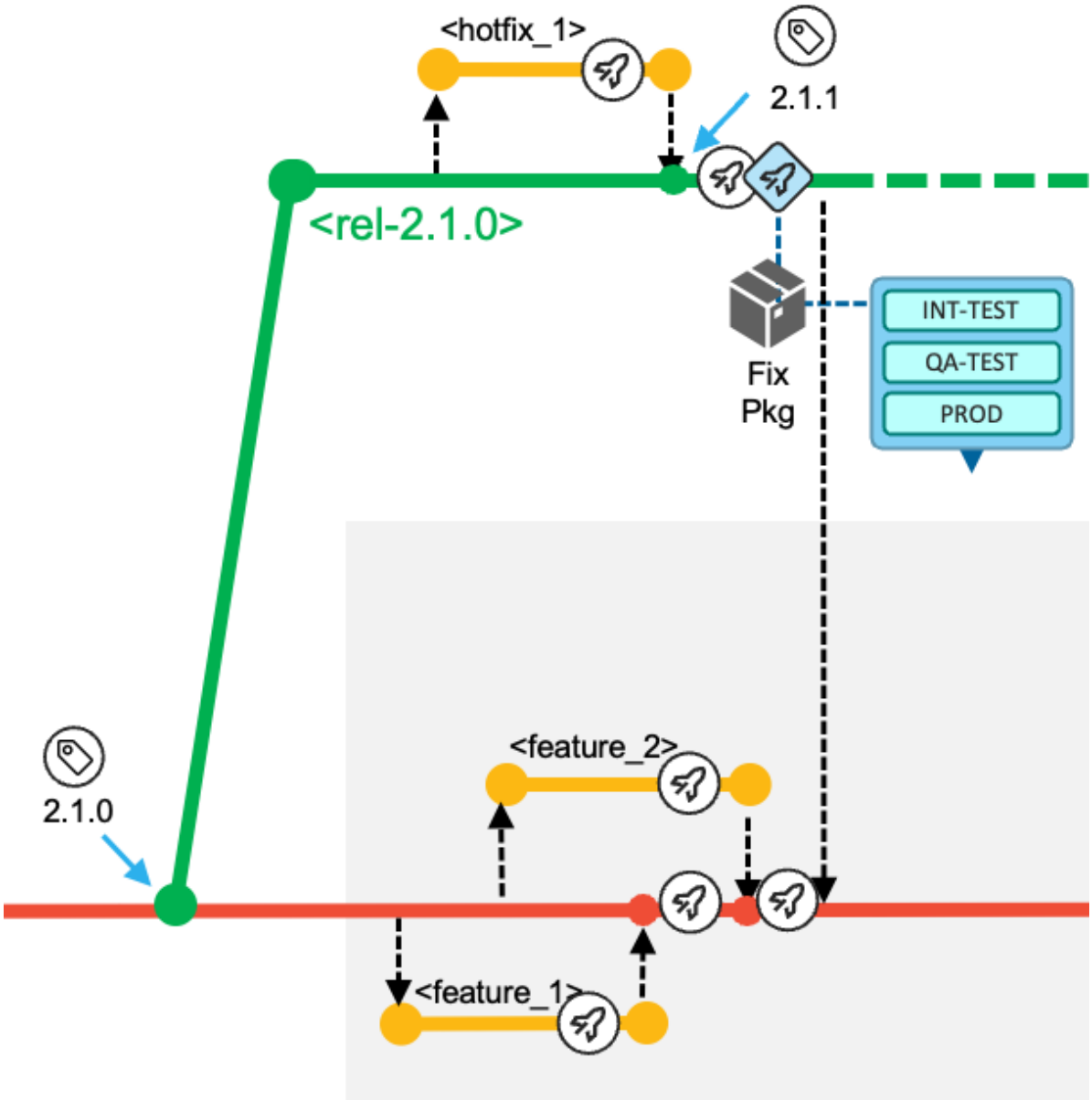
3.3 Features for the next planned release

Release control activities:

7. Merge approved branches to `main` and build
8. When planned features have merged, build and tag Release Candidate package
9. Deploy Release Candidate to test environments
10. (*Release candidate can be superceded*)
11. Finalise Release package in Artifactory (or equivalent)
12. Tag commit from step 8 as the production release and deploy

3.4 Fix a production defect

- The process of urgent fixes for modules in the production environment follows the fix-forward approach, rather than rolling back the affected modules and reverting to the previous deployed state.
- The development team starts this process by creating a **release maintenance** branch of the git tag that represents the most recent release, that was deployed to production.



3.5 Fix a production defect

Developer activities resolving a production defect:

1. Creates the `release/rel-2.1.0` branch based on the existing Git tag
2. Creates a feature branch `hotfix/rel-2.1.0/urgent-fix` from the release maintenance branch.

3. Implement the fix locally
4. Build with DBB User Build
5. Commit into hotfix branch, push, build and test feature functionality
6. Create Pull Request for review and approval

3.6 Fix a production defect

Release control activities resolving a production defect:

7. Merge hotfix branch to release maintenance branch and build
8. Use release pipeline to create fix package
9. Deploy fix package to controlled test environments and test it
10. Deploy fix package to production
11. Tag commit from step 8 indicating the new version
12. Merge release maintenance branch into `main`.

3.7 Using the Epic branch workflow

Dealing with significant development initiatives such as:

- Updates in regulatory frameworks
- Start the development phase of *subsequent* releases

→ A process to work independently from the base workflow.

3.8 Using the Epic branch workflow

Developer activities for implementing significant development initiatives:

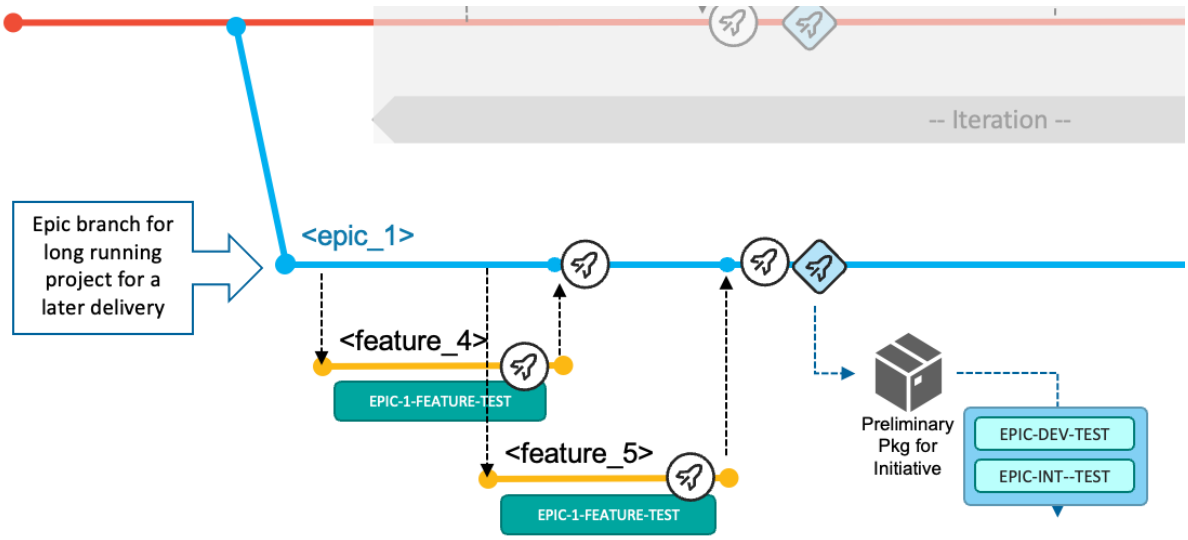
1. Creates the `epic/ai-fraud-detection` branch based on the existing Git tag
2. Creates a feature branch `feature/ai-fraud-detection/introduce-ai-model-to-mortgage-calculati` from the epic branch.
3. Code locally
4. Build with DBB User Build
5. Commit into feature branch, push, build and test feature functionality
6. Create Pull Request for review and approval to merge into the integration branch

3.9 Using the Epic branch workflow

Release activities for implementing significant development initiatives:

7. Merge feature branch to epic branch and build
8. Use release pipeline to create preliminary package
9. Deploy package to project test environment
10. Frequently synchronize with `main` into the epic branch to avoid making the merge of the `epic` branch into `main` difficult.
11. Decision **when** the changes on the epic branch should be released
12. Merge epic branch into `main` and deliver via the base workflow.

3.10 Epic branch workflow



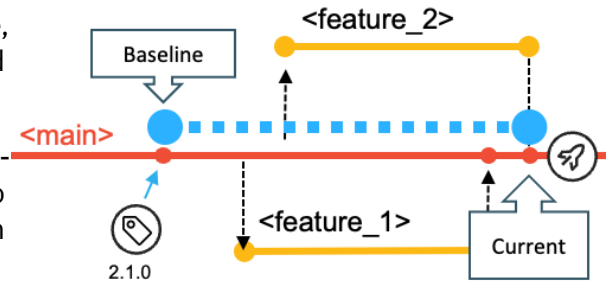
4 Pipeline implementations

4.1 Pipeline implementation conventions & principles

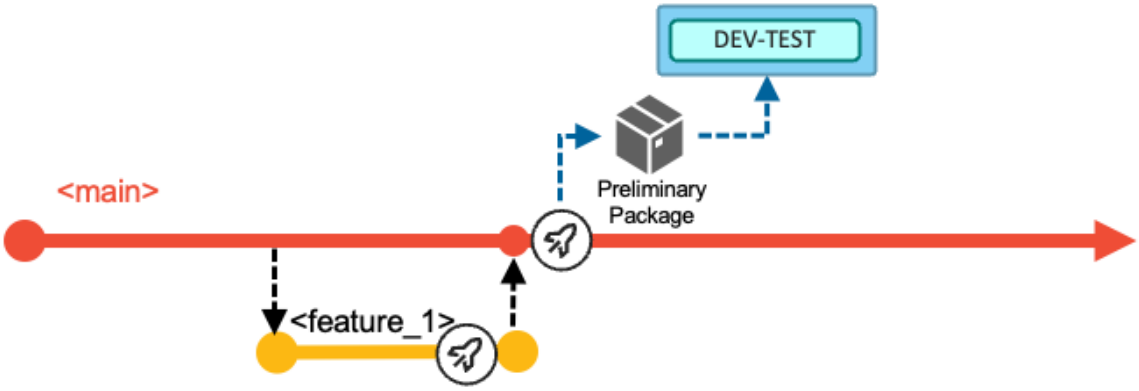
4.2 Basic Build pipeline characteristics

- (Typically) runs on each commit added to the history of a branch
- Can produce a preliminary package that is stored in a binary artifact repository
  - Can be deployed to test environments, but not production

- Git tags are used to calculate the changes for the next planned release, release maintenance packages and larger development activities.
- Packages are cumulative which is making it easier to get a test environment to the desired state (avoiding coordination of several incremental packages)
  - A new release candidate package supersedes preview release candidates.



- May include an automated step to deploy it into the DEV-TEST environment

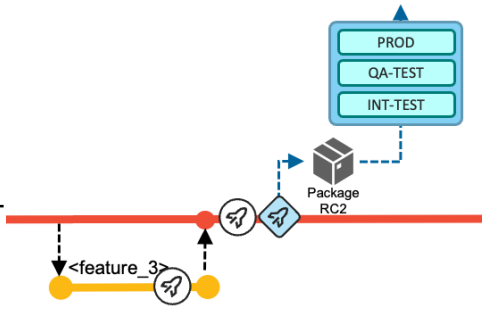
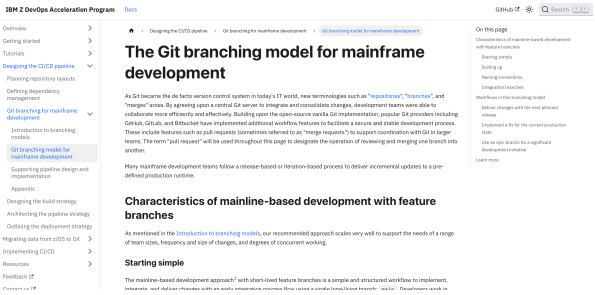


4.3 Release pipeline

5 Collateral

5.1 References

- Produces binaries optimized for performance
  - that can be deployed to test and production environments.
- (Typically) is manually requested by the development team when a level of stability is reached
- Produces a release candidate package that can be deployed to higher test environments.



- zDevOps adoption enablement website - Git branching for mainframe development
- Recent blog post - A no-baggage approach to new Mainframe development practices
- December 6th - Accelerating Git Migration Initiatives - Setting up an Azure pipeline for a mainframe application with the common backend scripts